

PROGRAMACIÓN III

Trabajo Práctico - CRUD Categorías + CRUD Productos

OBJETIVO GENERAL

Desarrollar una aplicación Full Stack conectando un frontend React + TypeScript con un backend FastAPI. El estudiante implementará el CRUD completo de Categorías desde la interfaz de usuario, integrando el backend mediante fetch nativo. Los CRUDs de Producto y ProductoCategoria serán construidos en el backend y demostrados mediante un archivo REST Client.

MARCO TEÓRICO

Concepto	Aplicación en el proyecto
React	Biblioteca para construir interfaces de usuario mediante componentes reutilizables.
TypeScript	Superset tipado de JavaScript para detectar errores en compilación y mejorar la legibilidad.
Vite	Herramienta de build moderna que inicializa proyectos React con TypeScript, reemplazando Create React App.
FastAPI	Framework Python de alto rendimiento para construir APIs REST. Permite definir endpoints, validar datos con Pydantic y exponer documentación automática en /docs.
Pydantic	Librería de validación de datos usada por FastAPI para definir modelos (schemas) con tipado estricto.
fetch nativo	API del navegador para realizar peticiones HTTP. Se usa con async/await para consumir la API REST desde el frontend.
useState	Hook de React para manejar estado local en un componente funcional (ej: lista de categorías, estado del modal).
useEffect	Hook de React para ejecutar efectos secundarios (ej: fetch al montar el componente para cargar datos).
Interface	Construcción de TypeScript que define la forma de un objeto (ej: Categoria, Producto).
Tailwind CSS	Framework de CSS utilitario. Estilos aplicados directamente en el JSX mediante clases predefinidas.
CORS	Mecanismo que permite al frontend (localhost:5173) comunicarse con el backend (localhost:8000) en dominios distintos.

CASO PRÁCTICO

Parte A — Backend con Fast Api

Estructura esperada del proyecto
fastapi-productos/

```
|
|
|— app/
|   |— __init__.py
|   |— main.py
|   |
|   |— core/
|       |— __init__.py
|       |— database.py
|   |
|   |— categoria/
|       |— __init__.py
|       |— router.py
|       |— model.py
|       |— schema.py
|       |— service.py
|   |
|   |— producto/
|       |— __init__.py
|       |— router.py
|       |— model.py
|       |— schema.py
|       |— service.py
|
|— .env
|— .gitignore
|— requirements.txt
|— README.md
```

a) Crear y activar un entorno virtual Python, crea archivo de dependencias e instalar las dependencias:

```
python -m venv .venv
source .venv/bin/activate    # macOS/Linux
.venv\Scripts\activate      # Windows
pip install -r requirements.txt
```

b) Definir los models y schemas.

Categoria	Producto	Producto_Categoria
id: int nombre: str descripcion: str	id:int nombre:str descripcion:str precio_base:str imagen_url:list[] disponible: bool	producto_id:int categoria_id:int

c) Implementar los endpoints de Categorías en `./categoria/router.py`, deberá de habilitar los CORS para poder consumir su API desde el Front-End:

(Código de ejemplo)

```
from fastapi import APIRouter
from fastapi.middleware.cors import CORSMiddleware

app = FastAPI()

app.add_middleware(
    CORSMiddleware,
    allow_origins=['http://localhost:5173'],
    allow_methods=['*'],
    allow_headers=['*'],
)
```

d) Implementar también los endpoints de Producto. Estos no necesitan conectarse al frontend — serán demostrados con REST Client. (El archivo `.http` debera ser entregado junto al proyecto)

e) Ejecutar el servidor:

```
python -m fastapi dev main.py
```

Parte B — Frontend con React + TypeScript

a) Inicializar el proyecto:

```
pnpm create vite@latest tp-categorias -- --template react-ts
cd tp-categorias
pnpm install
```

b) Instalar y configurar Tailwind CSS.

c) Definir el tipo en `src/types/categoria.ts`:

d) Crear los componentes en `src/components/` (.tsx):

- **Navbar.tsx** — barra de navegación con el nombre de la app.
- **CategoriaCard.tsx** — tarjeta que muestra una categoría (nombre + descripción). Recibe props tipados con la interface `Categoria`. Incluye botones Editar y Eliminar.
- **CategoriaList.tsx** — recibe `Categoria[]` por props y renderiza las tarjetas.
- **CategoriaModal.tsx** — formulario de alta/edición dentro de un modal. Se abre y cierra con `useState`. Llama a `onSubmit` al guardar.

e) En `App.tsx` implementar la lógica principal:

- `useState` para la lista de categorías, el estado del modal (abierto/cerrado) y la categoría seleccionada para edición.
- `useEffect` para cargar las categorías al montar el componente (`GET /categorias`).
- Funciones `handleCreate`, `handleUpdate`, `handleDelete` que usen `fetch` nativo hacia el backend.
- Pasar las funciones y el estado como props a los componentes hijos.

f) Verificar que el proyecto corre sin errores TypeScript:

```
pnpm dev
```

Ejemplo de pantalla FRON-TEND

Categorías [+ Añadir Categoría](#)

NÚMERO	NOMBRE	DESCRIPCIÓN	ACCIONES
1	Pizzas	Pizzas artesanales con masa fresca	Editar
2	Hamburguesas	Hamburguesas gourmet con ingredientes frescos	Editar
3	Bebidas	Gaseosas, jugos naturales y agua	Editar
4	Postres	Helados, tortas y dulces artesanales	Editar
5	Entradas	Empanadas, medialunas y snacks para compartir	Editar
6	Pastas	Pastas frescas con salsas caseras	Editar
7	pepe	Pizzas artesanales con masa fresca	Editar

Categorías [+ Añadir Categoría](#)

NÚMERO	NOMBRE	DESCRIPCIÓN	ACCIONES
1	Pizzas	Pizzas artesanales con masa fresca	Editar
2	Hamburguesas	Hamburguesas gourmet con ingredientes frescos	Editar
3	Bebidas	Gaseosas, jugos naturales y agua	Editar
4	Postres		Editar
5	Entradas		Editar
6	Pastas		Editar
7	pepe		Editar

Editar Categoría

Nombre de la categoría

Descripción

[Cancelar](#) [Guardar](#)

CONCLUSIONES ESPERADAS

Al finalizar el trabajo práctico, el estudiante debe demostrar:

- **Integración Full Stack:** El frontend React consume la API FastAPI mediante fetch nativo. El CRUD de Categorías funciona de extremo a extremo.
- **Modal con useState:** El formulario de alta y edición está dentro de un modal controlado con estado booleano.
- **Hooks aplicados correctamente:** useState y useEffect utilizados en App.tsx. Los demás componentes son funcionales puros sin hooks.
- **Tipado TypeScript:** Interfaces Categoría y Producto definidas. Props tipados en todos los componentes .tsx.
- **Backend REST documentado:** Endpoints de Producto y ProductoCategoría funcionando y demostrados en el archivo .http.
- **CORS configurado:** El backend permite peticiones desde el origen del frontend.

RÚBRICA DE EVALUACIÓN

Criterio	Descripción	Puntos
CRUD Categorías – Frontend	Listar, crear, editar y eliminar categorías desde React usando fetch nativo y useState/useEffect.	30
Modal funcional	Modal que se abre para crear y editar categorías, controlado con useState. Se cierra al guardar o cancelar.	15
CRUD Categorías – Backend	Endpoints FastAPI GET /categorias, POST, PUT/{id}, DELETE/{id} con Pydantic y persistencia en lista/DB.	20
CRUD Productos y relación – REST Client	Endpoints de Productos y ProductoCategoría demostrados con REST Client (.http). No requiere UI.	15
Tipado TypeScript	Interfaces Categoría y Producto tipadas. Props tipados en todos los componentes.	10
Estructura y buenas prácticas	Carpetas components/ y types/. Tailwind CSS aplicado. Sin errores de TS en consola.	10

Modalidad de Entrega

- Eliminar la carpeta node_modules antes de comprimir
- Proyecto Frontend comprimido en un archivo .zip
- Proyecto Backend comprimido en un archivo .zip

RECURSOS ADICIONALES

Documentación oficial:

- React: <https://react.dev/>
- TypeScript: <https://www.typescriptlang.org/docs/>
- Tailwind CSS: <https://tailwindcss.com/docs/>
- Vite: <https://vitejs.dev/>
- FastAPI: <https://fastapi.tiangolo.com/>